The Journal of Open Source Software

samplesheet-parser: A multi-vendor parser, validator, and converter for sequencing sample sheets

Chaitanya Krishna Kasaraneni [1]

1 Independent Researcher

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 30 August 2026
Published: unpublished

Summary

Every high-throughput sequencing run begins with a *sample sheet*: a small CSV file that maps each sample to its index barcodes and tells the demultiplexer how to split raw signal into per-sample reads. Despite being the linchpin of every run, the sample sheet has no single standard. Illumina alone ships two mutually incompatible layouts, the legacy IEM V1 format consumed by `bcl2fastq` (Illumina, Inc., 2020) and the modern BCLConvert V2 format (Illumina, Inc., 2023), and non-Illumina platforms such as the Element AVITI (Arslan et al., 2024) introduce their own run-manifest dialect.

`samplesheet-parser` is a dependency-free Python library and command-line tool that parses, validates, converts, diffs, merges, splits, filters, and programmatically writes sequencing sample sheets behind one consistent interface. It auto-detects the file format across vendors and exposes the same `samples()` and `index_type()` API regardless of origin, so downstream code never has to branch on format. Beyond structural parsing, it performs index-integrity checks (character set, length, duplicates, and wildcard-aware Hamming distance), decodes `OverrideCycles` strings to locate unique molecular identifiers (Smith et al., 2017), and models each instrument's optical detection chemistry to flag index pools that are not color balanced.

The Journal of Open Source Software

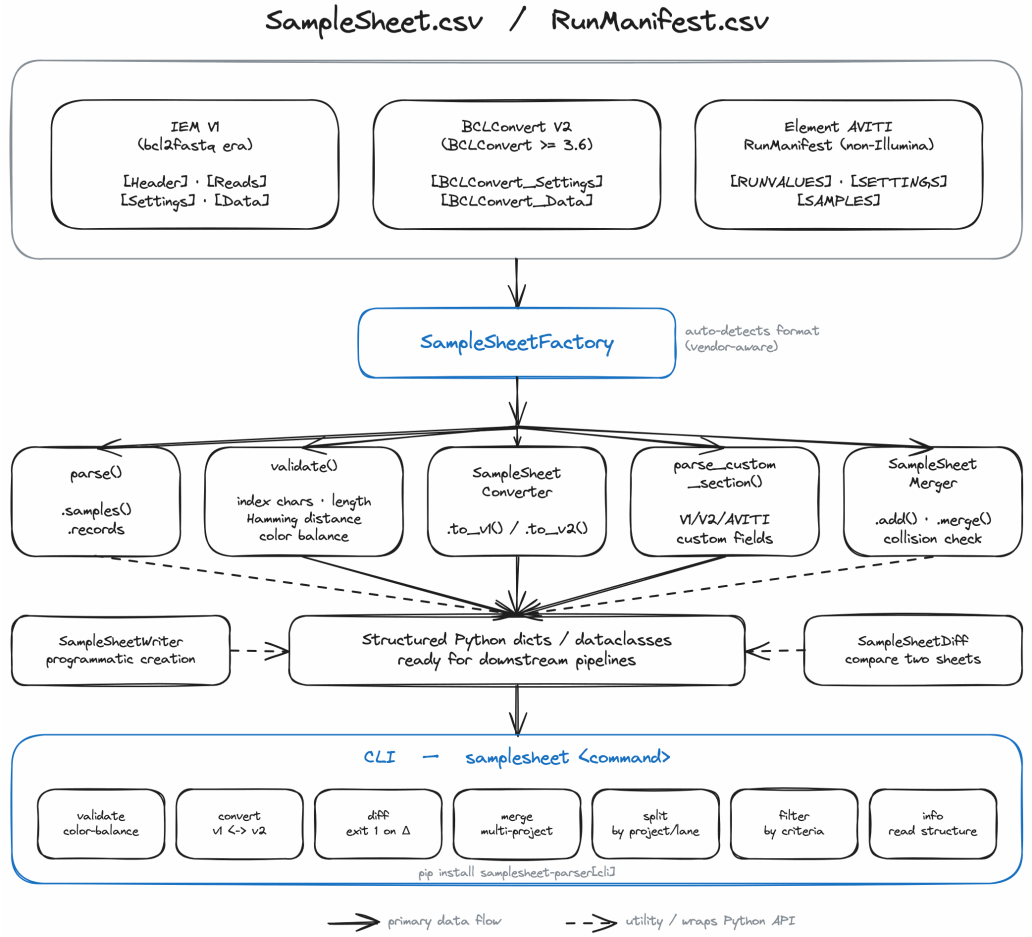

Figure 1: Architecture of `samplesheet-parser`. A single factory auto-detects the input format, either Illumina V1/V2 or an Element AVITI run manifest, and routes it to a parser that exposes a shared interface; validation, conversion, diffing, merging, and writing all operate on the common representation.

Statement of need

Core facilities and sequencing labs routinely operate mixed instrument fleets, producing sample sheets in several incompatible formats that must be validated before a run is committed. A single error in any of these files, whether a duplicated index, a barcode collision, or an index design that produces no optical signal, can invalidate an entire flow cell after the reagents and machine time have already been spent. The most damaging errors are precisely those that are *syntactically valid* yet fail on the instrument, so structural validation alone is not enough: laboratories need tooling that understands each platform's formats and its optical chemistry, and that presents a single interface across a heterogeneous fleet.

State of the field

Existing open-source tools address only a slice of this problem. The widely used `sample-sheet` library (Valentine, 2023) reads and writes the IEM V1 format but does not support BCLConvert V2, while `samshee` (Leibniz Institute for Immunotherapy (LIT), Regensburg, 2025) validates V2 sheets against the Illumina schema but does not handle V1, convert between formats, or address non-Illumina vendors. In both cases the validation is *structural*: it confirms that fields are present and well-formed, but cannot catch design errors that will fail on the instrument,

such as an index pool that produces no optical signal on two-channel chemistry. `samplesheet-parser` is, to our knowledge, the only open-source tool that combines cross-vendor parsing, bidirectional V1/V2 conversion, and instrument-aware optical color-balance validation in one interface.

Software design

`samplesheet-parser` is organized around a single auto-detecting factory and a structural `Protocol` that every parser satisfies. Three design decisions follow from this:

1. **Cross-format support and conversion.** The factory parses IEM V1 and BCLConvert V2 sheets through a shared interface and converts between them where possible, with explicit warnings when V2-only fields cannot be represented in V1. This lets a lab standardize tooling across old and new instruments without rewriting pipelines (Figure 1).
2. **Optical color-balance validation.** Two-channel instruments (for example NextSeq and NovaSeq X) read the base `G` as a dark, no-dye signal; an index cycle in which the entire pool reads `G` therefore registers no light and silently fails to demultiplex, an error that index-distance checks cannot detect. The library models the four-, two-, and one-channel chemistries of Illumina instruments, as well as the avidity chemistry of the Element AVITI, and scores an index pool cycle by cycle. It offers a `vendor_faithful` mode that encodes each platform's published rule and a stricter `conservative` mode, moving a class of run-ending mistakes from post-hoc troubleshooting to pre-run validation.
3. **Multi-vendor extensibility.** The same factory recognizes Element AVITI `RunManifest.csv` files and parses them through the identical interface, mapping manifest fields onto the shared sample schema. Because AVITI uses an avidity chemistry (Arslan et al., 2024) in which each base carries its own dye and there is no dark base, the color-balance model applies the appropriate rules automatically. Because the parser is built around a structural `Protocol`, support for additional platforms can be added without modifying the core.

The tool is usable both as a library, for embedding in laboratory information management systems and demultiplexing pipelines, and as a Typer-based command-line application (Ramírez, 2020) with machine-readable JSON output for continuous-integration gating of sample sheets before sequencing. It has no runtime dependencies, is fully typed (`mypy --strict`), and is tested with an automated suite of over 750 unit tests at roughly 98% line coverage.

Research impact

By moving optical color-balance checking from post-hoc troubleshooting to pre-run validation, `samplesheet-parser` targets a class of failures that waste sequencing reagents and instrument time, a concrete cost for the core facilities and multi-instrument labs that are its intended users. The software is distributed through the channels those users already rely on: it is released on the Python Package Index (PyPI) and packaged for BioConda with a corresponding BioContainers image, and it ships a set of nf-core-style Nextflow modules (validate, convert, diff, merge, split, filter, and info) that have been contributed to and accepted into the community `nf-core/modules` repository, so the same operations can be dropped into existing Nextflow workflows. Each release is archived on Zenodo with a citable DOI. Together these integrations give the tool a credible path to adoption within established sequencing-analysis infrastructure.

AI usage disclosure

Portions of the code, tests, and documentation were developed with the assistance of generative AI tools. All such output was reviewed, tested, and verified by the author, who takes

responsibility for the final content.

Acknowledgements

The author thanks the open-source bioinformatics community for the public format documentation and reference tools that informed this work, and the nf-core community for review of the accompanying Nextflow modules.

References

Arslan, S., Garcia, F. J., Guo, M., & others. (2024). Sequencing by avidity enables high accuracy with low reagent consumption. *Nature Biotechnology*, *42*, 132–138. https://doi.org/10.1038/s41587-023-01750-7

Illumina, Inc. (2020). *bcl2fastq2 conversion software v2.20 software guide*. https://support.illumina.com/sequencing/sequencing_software/bcl2fastq-conversion-software.html

Illumina, Inc. (2023). *BCL convert software guide*. https://support.illumina.com/sequencing/sequencing_software/bcl-convert.html

Leibniz Institute for Immunotherapy (LIT), Regensburg. (2025). *Samshee: A schema-agnostic parser and writer for illumina sample sheets v2 and similar documents*. https://github.com/lit-regensburg/samshee

Ramírez, S. (2020). *Typer: Build great CLIs. Easy to code. Based on python type hints*. https://github.com/fastapi/typer

Smith, T., Heger, A., & Sudbery, I. (2017). UMI-tools: Modelling sequencing errors in unique molecular identifiers to improve quantification accuracy. *Genome Research*, *27*(3), 491–499. https://doi.org/10.1101/gr.209601.116

Valentine, C. (2023). *Sample-sheet: Parse illumina sample sheets with python*. https://github.com/clintval/sample-sheet